
Guia de Perfiles

Lite & Full

Creacion, migracion y combinacion de aplicaciones
offline-first con el stack SKILLS-AHAGUILERA

Caracteristica	Lite	Full
Apertura	Doble clic index.html	Bun --compile .exe
Runtime	Cualquier navegador	Windows (sin dependencias)
ES6 imports	No (file:// bloquea CORS)	Si (dentro de src/)
Base de datos	Dexie (IndexedDB)	Dexie + SQLite opcional
Cifrado	CryptoJS	CryptoJS
Librerias	assets/js/libs/ (curl/zip)	node_modules/ (bun add)
Tamano aprox.	~5 MB	~50 MB (.exe empaquetado)
Servicio IA	FlexSearch + estadisticas	+ PDF/DOCX/XLSX/CSV + QA
Distribucion	GitHub Pages + ZIP	GitHub Pages + Release .exe

Documento v1.0 — Junio 2026

SKILLS-AHAGUILERA — Stack offline-first profesional

Tabla de Contenido

1. Perfil Lite: Crear una app desde cero
 2. Perfil Full: Crear una app desde cero
 3. Migrar de Lite a Full
 4. Combinar ambos perfiles
 5. Reescribir una app existente con SKILLS-AHAGUILERA
- Anexo A** Comparativa de comandos
- Anexo B** Solucion de problemas comunes

■ 1. Perfil Lite

El perfil **Lite** genera aplicaciones que se abren con doble clic en **index.html**, sin servidor, sin build step, sin dependencias de sistema. Ideal para prototipos, apps educativas, herramientas internas y despliegue inmediato via GitHub Pages.

Requisitos: Solo un navegador moderno (Chrome, Edge, Firefox). No requiere Node.js, Bun ni ningun runtime.

Paso a paso

Paso 1: Iniciar el pipeline

Ejecuta el comando **/new** en OpenCode. El prompt-inicial te preguntara: nombre de la app, tipo de app (ej: 'gestion de tareas', 'inventario'), descripcion y perfil. Responde **Lite**.

Paso 2: Setup automatico

El skill **setup-init** descargara todas las librerias necesarias en `assets/js/libs/` usando curl y un script .bat. Librerias incluidas: Alpine.js, Dexie, CryptoJS, DaisyUI, Tailwind CSS, Bootstrap Icons, Animate.css, ApexCharts, jsPDF, SheetJS, pako.

Paso 3: Definir la especificacion

El skill **spec-creator** guiara una sesion interactiva para definir funcionalidades, modelo de datos, user journeys y criterios de prueba. El resultado se guarda en `specs/[app].md`.

Paso 4: Generar codigo

El skill **code-generator** genera los modulos uno por uno: core (db.js, crypto.js, ui.js, theme.js, app.js) y modulos funcionales en `modules/[modulo]/`. Todo en JavaScript plano, sin imports ES6, apto para file://.

Paso 5: Validar la app

Ejecuta **/test**. El skill **validation-offline** corre 3 fases: validacion estatica (cumplimiento de reglas), guia DevTools (consola, storage, lighthouse) y tests E2E con Playwright. Genera reporte en `docs/validacion-[app].md`.

Paso 6: (Opcional) Anadir IA Jutia

Responde **/ia** y elige perfil **Lite**. Se genera el modulo IA con busqueda FlexSearch en memoria, estadisticas y predicciones simples. Todo offline.

Paso 7: Publicar

Ejecuta **/deploy**. **deployment-jigue** hace: commit con mensaje descriptivo, push a GitHub, empaquetado ZIP de toda la app en `dist/`, y activacion de GitHub Pages.

Tip: Despliegue rapido: Si ya tienes el repo configurado con Pages, con solo **/deploy** la app queda publicada en minutos. El ZIP generado tambien sirve para distribucion manual (USB, intranet, email).

Estructura de archivos generada

```
mi-app/  
index.html ← Punto de entrada (doble clic)  
project.config.js ← Configuración del proyecto  
assets/  
js/  
libs/ ← Librerías descargadas (Alpine, Dexie, etc.)  
core/  
db.js ← Capa de datos (Dexie)  
crypto.js ← Cifrado (CryptoJS)  
ui.js ← Componentes UI compartidos  
theme.js ← Tema claro/oscuro  
app.js ← Inicializador  
css/  
app.css ← Estilos personalizados  
modules/  
[modulo1]/  
module.js  
module.html  
...  
specs/  
mi-app.md ← Especificación técnica  
docs/  
validacion-mi-app.md ← Reporte de validación  
test_results.json  
dist/  
mi-app.zip ← App empaquetada para distribuir
```

■ 2. Perfil Full

El perfil **Full** genera aplicaciones compiladas con **Bun --compile**, produciendo un ejecutable `.exe` autocontenido (~50 MB) que no requiere runtime ni dependencias. Ideal para apps profesionales, distribucion a clientes, entornos sin navegador moderno y casos de uso que requieren SQLite, ingesta de documentos o modelos de IA locales.

Requisitos: Bun (<https://bun.sh>) instalado en el sistema de desarrollo. Windows 10+ para el `.exe` compilado.

Paso a paso

Paso 1: Iniciar el pipeline

Ejecuta **/new**. Responde al prompt-inicial con el nombre, tipo, descripcion y perfil: **Full**. El orquestador detectara que todas las fases usaran la variante Full.

Paso 2: Setup automatico

setup-init ejecutara `bun init` para crear `package.json` y luego `bun add` para instalar las dependencias (Dexie, CryptoJS, Alpine, DaisyUI, etc.) en `node_modules/`. Tambien crea la estructura `public/` para los archivos frontend y `src/index.js` como punto de entrada del backend Bun.

Paso 3: Definir especificacion

Igual que en Lite: **spec-creator** guia la sesion interactiva. El modelo de datos puede incluir opcionalmente tablas SQLite para datos pesados, ademas de las tablas Dexie para el frontend.

Paso 4: Generar codigo

code-generator genera ~95% del codigo identico a Lite en `public/` (los modulos Alpine + HTML). Adicionalmente genera `src/index.js` con el backend Bun: servidor HTTP estatico, manejo de archivos, SQLite (opcional) y sincronizacion.

Paso 5: (Opcional) Anadir IA Jutia

Responde **/ia** y elige **Full**. Ademas de FlexSearch, se agregan: ingesta de PDF (`pdf.js`), DOCX (`mammoth.js`), XLSX (`SheetJS`), CSV y Markdown. Incluye `Transformers.js` con dos modelos: MiniLM-L6 (embeddings, ~80 MB) y BERT multilingual (QA, ~150 MB). Los modelos se descargan en `assets/models/` durante el setup.

Paso 6: Compilar y validar

Ejecuta **/test**. La validacion incluye ademas verificacion de que los imports en `src/` son correctos y que `public/` no contiene imports (regla de compliance).

Paso 7: Publicar

Ejecuta **/deploy**. **deployment-jigue** hace: commit, push, compilacion con `bun build --compile ./src/index.js --outfile dist/app.exe`, subida del `.exe` como Release asset a GitHub, y activacion de GitHub Pages para la version web (`public/`).

Tip: .exe portable: El .exe compilado con Bun incluye el runtime y todas las dependencias. El usuario final solo necesita descargar el archivo y ejecutarlo. No requiere Bun, Node.js ni ningún runtime instalado.

Estructura de archivos generada

```
mi-app/  
public/  
index.html ← Frontend (identico a Lite)  
project.config.js  
assets/  
js/  
core/ ← db.js, crypto.js, ui.js, theme.js, app.js (identicos a Lite)  
modules/ ← Modulos Alpine (identicos a Lite)  
src/  
index.js ← Backend Bun: servidor HTTP + SQLite + sync  
package.json  
node_modules/ ← Dependencias npm  
assets/  
models/ ← Modelos Transformers.js (Full IA)  
specs/  
mi-app.md  
docs/  
validacion-mi-app.md  
dist/  
app.exe ← Ejecutable compilado (~50 MB)
```

■ 3. Migrar de Lite a Full

Ya tienes una app funcionando con perfil Lite (doble clic en index.html). Quieres migrarla a Full para obtener: ejecutable .exe, backend Bun, SQLite opcional e IA completa con ingesta de documentos. El proceso es sencillo porque ~95% del código frontend es idéntico.

Proceso de migración

Paso 1: Actualizar project.config.js

Cambia perfil: 'lite' a perfil: 'full'. Esto le indica a todos los skills que trabajen en modo Full.

Paso 2: Ejecutar setup-init en modo Full

Ejecuta `/setup`. `setup-init` detectará que ya existe la estructura Lite y añadirá solo lo que falta: `src/index.js`, `package.json`, `node_modules/`. No modificará tus módulos existentes.

Paso 3: Reubicar assets si es necesario

Si tu app Lite usa librerías en `assets/js/libs/`, el `setup Full` las duplicará vía `bun add`. Puedes migrar las referencias en `index.html` de `assets/js/libs/alpine.js` a `node_modules/alpinejs/...`, o mantener ambas. El skill `code-generator` maneja esto automáticamente.

Paso 4: Compilar y probar

Ejecuta `bun build --compile ./src/index.js --outfile dist/app.exe`. Prueba que el `.exe` funcione correctamente y que el frontend se sirva desde el servidor embebido de Bun.

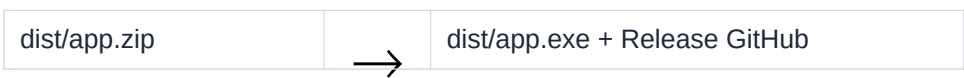
Paso 5: Actualizar el deploy

Ejecuta `/deploy`. Ahora `deployment-jig` empaquetará el `.exe` y lo subirá como Release asset, además de mantener GitHub Pages para la versión web.

Importante: La migración NO rompe la compatibilidad Lite. Tu `index.html` sigue funcionando con doble clic. La app Full simplemente añade capacidades nuevas sin eliminar las existentes. Puedes mantener ambos perfiles en paralelo.

Diagrama de migración

Lite		Full
index.html (doble clic)	■	public/index.html (idéntico)
assets/js/core/*.js	■	public/assets/js/core/*.js (idéntico)
modules/*/	■	public/modules/*/ (idéntico)
(no existe)	+	src/index.js + package.json + node_modules/
(no existe)	+	assets/models/ (modelos IA)



■ 4. Combinar ambos perfiles

El stack esta disenado para que ambos perfiles coexistan en el mismo proyecto. El 95% del codigo frontend es compartido, y puedes servir la misma app tanto como pagina web (Lite) como ejecutable de escritorio (Full).

Escenarios de combinacion

4.1 Desarrollo hibrido

Usa **Lite** durante el desarrollo: abres index.html con doble clic, iteracion rapida, sin esperar compilacion. Cuando la funcionalidad esta lista, compilas con **Full** para distribucion. No necesitas cambiar nada en el codigo.

```
# Flujo de trabajo recomendado:
1. Editas modulos en modules/ o public/
2. Pruebas al instante abriendo public/index.html (Lite)
3. Cuando esta listo: bun build --compile (Full)
4. Distribuyes el .exe
```

4.2 App hibrida: web + escritorio

Tu app se despliega en **GitHub Pages** para acceso web inmediato (version Lite) y simultaneamente publicas un **.exe** en GitHub Releases para quienes prefieran una aplicacion de escritorio. deployment-jigue maneja ambos despliegues.

Resultado: Mismo codigo, dos entregables: tus usuarios pueden elegir entre abrir la app en el navegador (sin instalar nada) o descargar el ejecutable para uso offline completo con capacidades adicionales (SQLite, IA).

4.3 Diferenciacion por modulo

Puedes tener modulos que solo se activen en Full. Ejemplo: un modulo de ingesta de PDFs que no tiene sentido en el perfil Lite (porque requiere mammoth.js y Transformers.js). Usa project.config.js para controlar que modulos se generan segun el perfil.

```
// project.config.js - Configuracion condicional
{
  "perfil": "full",
  "modulos": {
    "inventario": { "perfiles": ["lite", "full"] },
    "reportes": { "perfiles": ["lite", "full"] },
    "ia-lectura": { "perfiles": ["full"] }, // Solo Full
    "sync-cloud": { "perfiles": ["full"] } // Solo Full
  }
}
```

4.4 Mantenimiento unificado

Al compartir ~95% del código, cualquier mejora en un módulo se refleja automáticamente en ambos perfiles. Las únicas diferencias están en:

- Método de carga de librerías (assets/js/libs/ vs node_modules/)
- Punto de entrada (index.html directo vs servido por Bun)
- Backend (no existe en Lite vs src/index.js en Full)
- IA (FlexSearch solo vs FlexSearch + Transformers)
- Empaquetado (ZIP vs .exe)
- Base de datos (Dexie solo vs Dexie + SQLite opcional)

Matriz de compatibilidad

Componente	Lite	Full	Compartido
Modulos Alpine (UI)	✓	✓	✓ 95%
core/db.js	✓	✓	✓ identico
core/crypto.js	✓	✓	✓ identico
core/ui.js	✓	✓	✓ identico
IA (FlexSearch)	✓	✓	✓ base identica
IA (Transformers)	✗	✓	✗ solo Full
SQLite	✗	✓ opcional	✗ solo Full
src/index.js (backend)	✗	✓	✗ solo Full
Distribucion ZIP	✓	✓	✓ ambos
Distribucion .exe	✗	✓	✗ solo Full
GitHub Pages	✓	✓	✓ ambos

■ 5. Reescribir una app existente con SKILLS-AHAGUILERA

Tienes una aplicacion web funcionando (React, Vue, jQuery, HTML plano, etc.) y quieres reescribirla usando el stack offline-first de SKILLS-AHAGUILERA. El proceso transforma tu app existente en una aplicacion moderna, offline-first, sin CDNs, sin build step (o con Bun --compile si eliges Full), manteniendo las funcionalidades originales.

Beneficios de la reescritura: App offline-first (funciona sin internet), zero dependencias externas en runtime, cifrado local de datos sensibles, distribucion via doble clic o .exe, y un codigo base unificado mantenible con OpenCode.

Fases de reescritura

Fase A: Auditoria de la app existente

Antes de generar codigo nuevo, analiza la app actual para entender que funcionalidades preservar. No se trata de copiar codigo, sino de capturar el **comportamiento** y la **experiencia de usuario** existente.

- Inventario de pantallas: lista cada vista/pagina/modulo que tiene la app actual.
- Identifica las **entidades de datos** que maneja (ej: usuarios, productos, pedidos). Esto alimentara el modelo de datos Dexie.
- Observa los **flujos de usuario**: que hace el usuario desde que abre la app hasta que completa cada tarea? Esto alimentara los User Journeys en spec-creator.
- Documenta **reglas de negocio** importantes: calculos, validaciones, permisos logicos.
- Toma capturas de pantalla de referencia para mantener la paridad visual.
- Identifica dependencias externas: APIs, CDNs, librerias que habria que reemplazar por equivalentes offline (Dexie en vez de fetch, CryptoJS en vez de Web Crypto, etc.).

Fase B: Elegir perfil y configurar

Con la auditoria en mano, decide que perfil se adapta mejor:

- **Lite** si la app original era HTML/JS plano sin backend, o si necesitas compatibilidad maxima (doble clic, GitHub Pages).
- **Full** si la app original tenia backend (Node, Python, PHP, etc.) y necesitas SQLite, ingesta de documentos o IA con Transformers.
- **Lite + Full combinado** si quieres desarrollo rapido con doble clic y distribucion profesional con .exe.

Ejecuta **/new** y responde con el perfil elegido. Luego **/setup** para preparar la estructura y librerias.

Fase C: Especificacion desde el codigo existente

Ejecuta **/spec**. Usa los resultados de la auditoria para responder las preguntas de spec-creator. Cosas a tener en cuenta:

- Describe las funcionalidades en terminos de **lo que hace**, no de **como lo hace**. El spec-creator creara la implementacion optima para el stack offline-first.
- Proporciona el modelo de datos existente como referencia. spec-creator lo adaptara al modelo Dexie (con SQLite adicional si es Full).
- Si la app original usaba **fetch()** o **Axios** para datos, indicaselo a spec-creator: el reemplazo sera Dexie (lectura local) con sincronizacion diferida.
- Si la app original usaba **Web Crypto** o **bcrypt**, el reemplazo sera CryptoJS (compatible con file://).
- Si la app original tenia **autenticacion** (login, sesiones), describela: SKILLS-AHAGUILERA implementa checkSession() local con IndexedDB + verificacion en background si aplica.

Fase D: Generacion de codigo modular

Ejecuta **/build**. code-generator generara los modulos uno por uno, en orden. Por cada modulo generado:

- Compara visualmente con la captura de pantalla de la app original.
- Ajusta detalles de UI en `module.html` si es necesario (colores, textos, disposicion de elementos).
- Verifica la logica de negocio en `module.js`.
- Confirma que pase la validacion de **stack-compliance-guard**.

Tip: Reescritura progresiva: No necesitas generar todos los modulos de golpe. Puedes empezar con los modulos criticos, validarlos, y luego anadir el resto. Cada modulo es independiente.

Fase E: Migracion de datos

Si la app existente tiene datos locales (LocalStorage, SQLite, archivos JSON), puedes migrarlos a Dexie:

- Exporta los datos de la fuente original a JSON.
- Usa el helper de importacion en `core/db.js` para cargar los datos en IndexedDB con `bulkAdd()`.
- Si la migracion es grande, ejecutala en un solo bloque con `Promise.all()` para mantener la UI responsiva.
- Verifica que los datos migrados sean correctos abriendo la app y revisando cada modulo.

Fase F: Validacion y refinamiento

Ejecuta **/test** para la validacion completa. Ademas de las pruebas automaticas, realiza esta checklist manual:

- **Paridad funcional:** cada funcionalidad de la app original existe en la nueva.
- **Paridad de datos:** los datos migrados son correctos y completos.
- **Rendimiento:** la app se siente igual o mas rapida que la original.
- **Offline-first:** desconecta internet y verifica que todo funcione.
- **Distribucion:** prueba el ZIP (Lite) o el .exe (Full) en un equipo limpio.

Fase G: Publicar

Ejecuta **/deploy** para commit, push, empaquetado y despliegue. La app reescrita queda publicada en GitHub Pages + ZIP (Lite) o .exe + Release (Full).

Ejemplo: Reescritura de una app jQuery a Lite

App original: Gestor de tareas hecho con jQuery + LocalStorage + Bootstrap 4 via CDN.

Fase	Que paso
Auditoria	3 pantallas (lista, detalle, formulario), entidad 'tarea', datos en LocalStorage, sin backend.
Perfil elegido	Lite (compatible con doble clic, sin backend).
Setup	/new → 'gestor-tareas' → Lite → /setup. Librerías descargadas en assets/.
Spec	/spec. Se definen: CRUD de tareas, búsqueda, filtros por estado, exportacion CSV.
Generacion	/build. Se generan modulos: tareas-lista, tareas-detalle, tareas-form. Cada uno se revisa contra la app original.
Migracion datos	Script unico: leer localStorage → JSON → bulkAdd() a Dexie. ~50 tareas migradas en milisegundos.
Validacion	/test. Se corrige un detalle de CSS. Todo pasa.
Publicacion	/deploy. App en GitHub Pages + ZIP listo para compartir.

Diferencia clave con 'Migrar de Lite a Full': La **Seccion 3** asume que YA tienes una app SKILLS-AHAGUILERA en perfil Lite y quieres cambiar a Full. Esta **Seccion 5** asume una app EXTERNA (React, jQuery, PHP, etc.) que quieres reescribir completa desde cero con el stack SKILLS-AHAGUILERA. Son procesos diferentes.

Anexo A: Comparativa de comandos

Comando	Lite	Full
/new	Pregunta perfil, crea estructura Lite	Pregunta perfil, crea estructura Full
/setup	curl + .bat descarga libs en assets/	bun init + bun add en node_modules/
/spec	spec-creator v4 con modelo datos Dexie	spec-creator v4 con modelo datos Dexie + SQLite opcional
/build	Genera modulos en modules/ + index.html	Genera modulos en public/ + src/index.js
/test	3 fases: estatico + DevTools + Playwright	4 fases: + verificacion imports src/
/ia	FlexSearch + stats + predicciones	+ pdf.js + mammoth.js + Transformers.js QA
/deploy	commit + push + Pages + ZIP dist/	commit + push + Pages + .exe + Release

Nota: Todos los comandos se ejecutan en OpenCode. El perfil se configura una vez al crear el proyecto en `project.config.js` y persiste para todas las fases.

Anexo B: Solucion de problemas comunes

Problema	Perfil	Solucion
El index.html no se abre en doble clic	Lite	Asegurate de que no haya imports ES6 (<code><script type="module"></code>). En file://, los imports CORS bloquean la carga. stack-compliance-guard detecta esto automaticamente.
Bun --compile falla	Full	Verifica que Bun este instalado (<code>bun --version</code>). Asegurate de que <code>src/index.js</code> no importe archivos fuera de <code>src/</code> que no esten en <code>node_modules</code> .
Los modelos de IA no se descargan	Full	Los modelos Transformers.js (~233 MB total) se descargan durante el setup. Si falla la descarga, ejecuta manualmente el script de descarga en <code>assets/models/download-models.js</code> . Requiere conexion a internet la primera vez.
La migracion de Lite a Full duplica archivos	Migracion	Es normal. El setup Full no elimina archivos Lite. Puedes eliminar <code>assets/js/libs/</code> si ya no lo necesitas, pero asegurate de que las referencias en <code>index.html</code> apunten a <code>node_modules/</code> .
GitHub Pages no se actualiza	Lite/Full	Verifica que el Action workflow <code>deploy-pages.yml</code> este presente y que la rama configurada sea main . Ejecuta !deploy nuevamente.
El .exe generado no abre la UI	Full	Asegurate de que <code>src/index.js</code> sirva correctamente los archivos estaticos de <code>public/</code> . Verifica que la ruta sea relativa al <code>.exe</code> .
La app reescrita no tiene todas las funciones de la original	Reescritura	Revisa la auditoria de la Fase A. Compara modulo por modulo. Puedes generar modulos adicionales con /build sin afectar los existentes. Si falta logica de negocio, actualiza la spec y regenera.
Los datos migrados no aparecen en la app nueva	Reescritura	Verifica que el script de migracion uso <code>bulkAdd()</code> correctamente. Abre DevTools > Application > IndexedDB para inspeccionar que los datos esten en las tablas correctas.